
EMPLOYMENT MANAGEMENT SYSTEM

Departments

Department CRUD, the details view, and the delete guard

DEPARTMENT MANAGEMENT FLOWS • 2 OCTOBER 2026

Contents

01	1. The files	3
02	2. Why reads are restricted here too	3
03	3. DTO differences you should notice	4
04	4. Flow A - List departments (paginated)	4
05	5. Flow B - Create a department	5
06	6. Flow C - Update a department	6
07	7. Flow D - Delete a department (and the guard rule)	6
08	8. Flow E - View a department with its employees	7
09	9. How a department is referenced from employees	8
10	10. Department endpoint summary	8

Departments group employees. Like employees, **all seven endpoints are manager-level only**.

1. The files

File	Role
<code>department/DepartmentController.java</code>	7 endpoints
<code>department/DepartmentService.java</code>	interface
<code>department/DepartmentServiceImp.java</code>	business rules + one private helper
<code>department/DepartmentRepository.java</code>	queries
<code>department/Department.java</code>	entity
<code>department/dto/DepartmentCreateReq.java</code>	create body, validated
<code>department/dto/DepartmentUpdateReq.java</code>	update body, <code>@Size</code> only
<code>department/dto/DepartmentResponse.java</code>	<code>{id, name, description}</code>
<code>department/dto/DepartmentDetailsResponse.java</code>	the above plus the employees
<code>department/mapper/DepartmentMapper.java</code>	4 conversion methods
Frontend	<code>pages/DepartmentsPage.jsx</code> , <code>DepartmentFormModal.jsx</code> , <code>DepartmentDetailsModal.jsx</code>

2. Why reads are restricted here too

```
// Security model:
// - ALL endpoints (READ + WRITE): ADMIN or MANAGER only
// Employees have no department access because DepartmentDetailsResponse
// exposes EmployeeBrief.salary to anyone who can reach it.
@PreAuthorize("hasAnyRole('ADMIN','MANAGER')")
```

The chain of reasoning: `GET /api/departments/{id}/employees` returns `DepartmentDetailsResponse`, whose nested `EmployeeBrief` includes `salary`. If an `EMPLOYEE` could reach that one endpoint, they could read every salary in the company by iterating `id = 1, 2, 3, ...`.

Note that `GET /api/departments` (names and descriptions only) is harmless in itself - it is restricted anyway, because the class-level annotation is the simplest rule that is guaranteed to be correct.

3. DTO differences you should notice

Create vs update validation

Field	Create	Update
<code>name</code>	<code>@NotBlank</code> + <code>@Size(max=100)</code>	<code>@Size(max=100)</code> only
<code>description</code>	<code>@NotBlank</code> + <code>@Size(max=255)</code>	<code>@Size(max=255)</code> only

The update DTO has **no** `@NotBlank`, because a partial update may omit a field. But `@Size` still applies to a value that is present, so sending `""` for `description` passes `@Size` (length $0 \leq 255$) and would overwrite the column with an empty string - even though the column is `nullable = false`. That is a real hole; the service helper partially covers the `name` case (below) but not `description`.

Note also that the update DTO uses **PUT** in the controller while employees use **PATCH**; behaviourally both are partial updates here because of the mapper's `IGNORE` strategy.

DepartmentDetailsResponse

```

private Long id;
private String name;
private String description;
private List<EmployeeBrief> employeeList;    // nested, only exists here

public static class EmployeeBrief {
    Long id, firstName, lastName, email, phone; LocalDate hireDate; BigDecimal salary;
}

```

`EmployeeBrief` exists **only** inside this DTO. It is a deliberately flat projection so that a department-with-employees payload is one JSON object with a list, rather than a `Department` entity containing `Employee` entities containing `Department` entities - which would recurse forever.

4. Flow A - List departments (paginated)

Endpoint: `GET /api/departments/paginated?page=0&size=10`

The path is `paginated`, not `paged` (employees use `/paged`). Two different names for the same idea - worth remembering so you do not call the wrong one.

Frontend (`http.js:64`):

```
export async function fetchDepartmentsPage(page) {
  return api(`/api/departments/paginated?page=${page}&size=10`);
}
```

`fetchDepartments()` (`http.js:61`) is the *other* endpoint - unpaged, `size=100` - used by the Dashboard, the Attendance filters and `EmployeeFormModal` for the dropdown.

Backend:

```
Page<Department> departments = departmentRepository.findAll(pageable);
return departments.map(departmentMapper::toDtoResponse);
```

Same shape as employees. `DepartmentsPage` reads `data?.content ?? []` and `data?.totalPages ?? 1`.

Note the mapping here is safe even though `Department.employeeList` is `LAZY`, because `DepartmentResponse` has no employee field - MapStruct never touches the lazy collection.

5. Flow B - Create a department

Endpoint: `POST /api/departments` -> `201`

```
if (departmentRepository.existsByNameIgnoreCase(createReq.getName())) {
  throw new DuplicateResourceException("Department name '" + createReq.getName() + "' already exists");
}
Department department = departmentMapper.toEntity(createReq);
Department savedDepartment = departmentRepository.save(department);
return departmentMapper.toDtoResponse(savedDepartment);
```

The message deliberately includes the offending name, so the user immediately knows which value collided.

There is **no** `.trim()` here, unlike `createEmployee`. So creating `" HR "` succeeds and stores `" HR "`, which is then not found by `findByNameIgnoreCase("HR")` (that matcher is case-insensitive but not whitespace-insensitive). That inconsistency between the two features is a real, explainable bug.

Frontend form (`DepartmentFormModal.jsx`) is simpler than the employee one: only `name` (input) and `description` (textarea), both `required`, both pre-filled from `department?.name || ""` when editing.

6. Flow C - Update a department

Endpoint: `PUT /api/departments/{id}`

```

Department department = departmentRepository.findById(id).orElseThrow(
    () -> new ResourceNotFoundException("This department with id " + id + " does not exist"));

validateDepartmentUpdate(updateReq, department);

departmentMapper.updateDepartment(updateReq, department);
return departmentMapper.toDtoResponse(department);

```

Note there is **no** `save()` call - dirty checking again, because the class is `@Transactional`.

The helper `validateDepartmentUpdate` (line 109)

```

if (updateReq.getName() != null) {
    if (updateReq.getName().trim().isEmpty()) {
        throw new InvalidInputException("Department name cannot be empty");
    }
    if (!department.getName().equalsIgnoreCase(updateReq.getName()) &&
        departmentRepository.existsByNameIgnoreCase(updateReq.getName())) {
        throw new DuplicateResourceException("Department name '" + updateReq.getName() + "' already
exists");
    }
}
}

```

Three gates, in order:

- 1. Only when a name was sent.** `null` means "don't touch the name".
- 2. Empty-after-trim check.** This exists precisely because `@NotBlank` is not on the update DTO: a whitespace-only name would otherwise pass `@Size` and reach the database. `" ".trim().isEmpty()` catches it. (A name of literally `" "` is also caught by the same line.)
- 3. Uniqueness, but only if it changed.** `!department.getName().equalsIgnoreCase(...)` prevents "Department name 'HR' already exists" when you edit the description of a department whose name you also resubmitted unchanged. This is the same guard as in `EmployeeServiceImp.updateEmployee`.

`InvalidInputException` -> **400 Bad Request** (not 409) - it is a bad value, not a conflict.

7. Flow D - Delete a department (and the guard rule)

Endpoint: `DELETE /api/departments/{id}` -> `204`

```

Department department = departmentRepository.findById(id)
    .orElseThrow(() -> new ResourceNotFoundException("Department not found with id " + id));

if (!department.getEmployeeList().isEmpty()) {
    throw new InvalidInputException("Cannot delete department with assigned employees");
}

departmentRepository.delete(department);

```

This is a business rule, not a database rule. `employees.department_id` is nullable, so the database *would* happily orphan employees by nulling their department. Instead the service refuses, returning **400** with a clear message.

Two details worth being able to explain:

- `department.getEmployeeList()` is a **lazy** collection, and reading `.isEmpty()` does **not** run a `SELECT`. Hibernate implements `isEmpty()` against the owning entity's known row count (from the `employeeList`'s `size`, available after loading the department), so it works here inside the transaction without an extra join.
- Contrast with employee delete, where the relationship is `cascade = ALL, orphanRemoval = true` and attendance rows are removed automatically. Departments have no cascade, so the guard is required.

Frontend: `window.confirm("Delete this Department")`, then the mutation's `onError` shows the `400` message in a red toast - which is the only place the user ever sees "Cannot delete department with assigned employees".

8. Flow E - View a department with its employees

Endpoint: `GET /api/departments/{id}/employees`

Screen: `DepartmentsPage.jsx:83` sets `details` from the clicked row, which renders

```
<DepartmentDetailsModal department={details} />.
```

`DepartmentDetailsModal` then issues its own request:

```

const { data, isLoading, error, isError } = useQuery({
  queryKey: ["department-employees", department.id],
  queryFn: () => fetchDepartmentEmployees(department.id),
});

```

So "View" is a **two-step interaction**: the row data (name + description) is already on screen, and the modal then loads the employee list. The query key includes the department id, so switching departments refetches.

Backend:

```

@Query("SELECT DISTINCT d FROM Department d " +
      "LEFT JOIN FETCH d.employeeList e " +
      "WHERE d.id = :id")
Optional<Department> findWithEmployeesById(@Param("id") Long id);

```

JAVA

- `LEFT JOIN FETCH d.employeeList` loads the employees **in the same query**. This is mandatory: `DepartmentDetailsResponse` needs the list, the collection is `LAZY`, and mapping happens inside the transaction. Without the fetch you would get N+1 queries (one per department) or a lazy-initialisation failure.
- `DISTINCT` collapses the row multiplication the join causes.

Then MapStruct maps `Department.employeeList` (a `List<Employee>`) into `List<EmployeeBrief>` by matching the shared field names. Fields on `Employee` that are not on `EmployeeBrief` are simply not copied.

The modal renders `employee.salary` with `Intl.NumberFormat` as USD, and shows `"-"` when the salary is `null`.

9. How a department is referenced from employees

The employee create/update API takes `departmentName` (a **string**), not a `departmentId`. The service resolves it with `departmentRepository.findByNameIgnoreCase(name)`. Consequences:

- The employee form dropdown is built from `fetchDepartments()` and uses `value={dep.name}` (`EmployeeFormModal.jsx:142`) - it submits the *name*.
- Renaming a department therefore **breaks** any client that cached the old name, and changes what the employee form must send. Referential integrity is by name, not by id. This is the single biggest design smell in the app; passing an id would be the fix.

10. Department endpoint summary

Method	Path	Body / Params	Success	Errors
POST	<code>/api/departments</code>	<code>DepartmentCreateReq</code>	<code>201</code> <code>DepartmentResponse</code>	400 validation, 409 name exists
GET	<code>/api/departments</code>	-	<code>200</code> <code>[DepartmentResponse]</code>	-
GET	<code>/api/departments/paginated</code>	<code>page</code> , <code>size</code> , <code>sort</code>	<code>200</code> <code>Page<DepartmentResponse></code>	-

Method	Path	Body / Params	Success	Errors
GET	/api/departments/{id}	-	200 DepartmentResponse	404
GET	/api/departments/{id}/employees	-	200 DepartmentDetailsResponse	404
PUT	/api/departments/{id}	DepartmentUpdateReq	200 DepartmentResponse	400 empty/changed name, 404, 409 name exists
DELETE	/api/departments/{id}	-	204 empty	404, 400 department has employees

All require `ROLE_ADMIN` or `ROLE_MANAGER`.